XA Transactions

Performance Considerations

Extended Architecture Transactions (or XA Transactions) can be used to group a series of operations from multiple transactional resources, such as VM, JMS or Database, into a single reliable global transaction.

The XA (eXtended Architecture) standard is an X/Open group standard which specifies the interface between a global transaction manager and local transactional resource managers. The XA protocol defines a 2-phase commit protocol which can be used to reliably coordinate and sequence a series of atomic operations across multiple servers of different types. Each local XA resource manager supports the A.C.I.D properties (Atomicity, Consistency, Isolation, and Durability) which help guarantee completion of a sequence of operations in the resource managed by the XA resource manager. Examples of transactional systems which often include local XA resource managers include databases, application servers, messaging queues, and transactional caches.

The global transaction manager is responsible for coordinating the transactional semantics of the global transaction between all the local XA transaction managers participating in the global transaction. To do this, each prepare and commit phase in the 2-phase commit is coordinated globally (often over the network) with each local XA resource manager.

The advantage of XA global transactions is that one global transaction manager can communicate and coordinate with multiple different transactional resources in a common and standard way. If any of the local resource managers report an error in the global transaction, the global transaction manager coordinates the rollback of any already prepared operations in each of the other local resource managers. For example, a global transaction might include several operations to one or more databases, as well as messages sent to a JMS server on one or more topics. If anything goes wrong in the transaction, the global transaction manager guarantees that every resource is reset to the state before the global transaction was started. This keeps developers from having to include complex rollback or recovery logic outside the application.

The disadvantage of XA global transactions is that they can add tremendous latency to your transactions, especially if the resources are distributed over slow or unreliable network connections, and it can be difficult to recover if there are physical failures of systems in the middle of a globally distributed transaction.

Performance Considerations

A global XA transaction is a reliable way of coordinating multiple XA resources, but it often increases latency and also requires additional resources such as an external transaction manager. A global transaction is only as reliable as the global transaction manager itself. For greater reliability, you need to use a transaction manager that also persists the state of each XA resource as the global transaction progresses through the prepare and commit phases, in case the transaction manager fails or loses network connectivity to any of the local XA resource managers during the global transaction.

Configuring your Resource to use XA Transactions

To configure the event sources that support XA transactions, refer to the Connectors documentation:

- JMS Connector
- IBM MQ Connector
- VM Connector
- Database Connector

Configuring a Try Scope to use XA Transactions

To configure a Try Scope to use XA Transactions, follow these steps:

1. Add the Try Scope from the Mule Palette.
2. Go to the **General** tab in the Try configuration panel.
3. Configure the Try Scope `Transactional Action` to `ALWAYS_BEGIN`, and the `Transaction Type` to `XA`.

See also Try Scope Concept for more information on how Try Scope handles transactions.

Using XA Transactions with Different Resources

Regardless of how the Transaction is started (either from a Transactional event source or from a Try Scope), any Connector Operation that supports Transactions may join the Transaction by setting its Transactional Action to `ALWAYS_JOIN` or `JOIN_IF_POSSIBLE`. XA Transactions are prepared to work with different resources, this means that operations from different resources can join the transaction. For example:

XML

```
<flow name="exampleFlow" >
        <try transactionalAction="ALWAYS_BEGIN" transactionType="XA">
                <set-payload value="Hello World"/>
                <vm:publish queueName="someVmQueue" config-ref="VM_Config"/>
                <jms:consume config-ref="JMS_Config" destination="someQueue"/>
                <db:insert config-ref="Database_Config">
                        <db:sql>${insertQuery}</db:sql>
                </db:insert>
        </try>
        <error-handler>
                <on-error-propagate enableNotifications="true" logException="true"/>
        </error-handler>
</flow>
```

If the `db:insert` operation fails, the transaction is rolled back before the error handler (`on-error-propagate`) is executed. Therefore, the message sent through the

`vm:publish` is not confirmed to be sent, and the message in the `jms:consume` is not actually consumed, so it is available next time to be consumed again.

In the following example, the error handler is changed to an `on-error-continue`:

XML

```
<flow name="exampleFlow" >
        <try transactionalAction="ALWAYS_BEGIN" transactionType="XA">
                <set-payload value="Hello World"/>
                <vm:publish queueName="someVmQueue" config-ref="VM_Config"/>
                <jms:consume config-ref="JMS_Config" destination="someQueue"/>
                <db:insert config-ref="Database_Config">
                        <db:sql>${insertQuery}</db:sql>
                </db:insert>
        </try>
        <error-handler>
                <on-error-continue enableNotifications="true" logException="true">
                        <jms:publish config-ref="ANOTHER_JMS_Config" destination="som
                </on-error-continue>
        </error-handler>
</flow>
```

If the `db:insert` operation fails, the transaction is not rolled back. Instead, the Transaction is committed after the `jms:publish` (which runs within the XA Transaction) executes. Therefore, the message sent using `vm:publish` is published, the message from `jms:consume` is consumed, and the message sent using `jms:publish` is also published.

Consider that the `jms:publish` inside the `on-error-continue` has another configuration for the connector. This is not required but is possible. With single source (local) Transactions, this is not a valid configuration, since all operations that join a Transaction must use the same Configuration to work as designed.

Nested Transactions

XA Transactions support nested transactions. You can create a new transaction when already executing within a transaction by using the Try Scope. Therefore, operations that run within the new transaction are accountable to it. If the nested transaction is rolled back, it does not mean that its parent transaction is also rolled back. After the nested transaction finishes (either with a commit or a rollback), the parent transaction continues its execution.

For example:

XML |

```xml
<flow name="exampleFlow" >
        <try transactionalAction="ALWAYS_BEGIN" transactionType="XA">
                <vm:publish queueName="someVmQueue" config-ref="VM_Config"/>
                <try transactionalAction="ALWAYS_BEGIN" transactionType="XA"/>
                        <jms:consume config-ref="JMS_Config" destination="someQueue"/
                        <db:insert config-ref="Database_Config">
                                <db:sql>${insertQuery}</db:sql>
                        </db:insert>
                        <raise-error type="APP:SOME"/>
                </try>
                <error-handler>
                        <on-error-continue/>
                </error-handler>
        </try>
</flow>
```

In the previous example, the first `try` Scope creates a transaction. Then, the second `try` Scope creates a second transaction. The `vm:publish` operation runs within the first transaction, the `jms:consume` and the `db:insert` run within the second transaction. When the error is raised, the second transaction is rolled back along with the `jms:consume` and the `db:insert` operations. However, the first `try` handles the error with an `on-error-continue`, so the first transaction commits along with the `vm:publish` operation.

Consider the following example:

XML

```xml
<flow name="exampleFlow" >
        <try transactionalAction="ALWAYS_BEGIN" transactionType="XA">
                <vm:publish queueName="someVmQueue" config-ref="VM_Config"/>
                <try>
                        <try transactionalAction="ALWAYS_BEGIN" transactionType="XA"/
                                <jms:consume config-ref="JMS_Config" destination="som
                                <raise-error type="APP:SOME"/>
                        </try>
                        <error-handler>
                                <on-error-continue/>
                        </error-handler>
                </try>
                <db:insert config-ref="Database_Config">
                        <db:sql>${insertQuery}</db:sql>
                </db:insert>
        </try>
</flow>
```

In this case, the second `try` scope does not create a transaction. It only provides error handling, so that the next operations can continue. The third `try` scope does create a transaction. Both `vm:publish` and `db:insert` run within the first transaction,

and `jms:consume` operation runs within the second one. When the error is raised, the second operation is rolled back. The error is propagated and handled by the `on-error-continue` of the second `try` scope. After that, the first `try` scope continues with its execution (the `db:insert` operation), and the first transaction is committed.

See Also

- Try Scope
- Using Bitronix to Manage Transactions
- Transaction Management

Back to top

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

Yes No

Legal | Terms of Service | Privacy | Trust | Contact | Responsible Disclosure | Cookie Settings
Your Privacy Choices

© Copyright 2026 Salesforce, Inc. All rights reserved. Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States